

AC-MOT: Optimization-Guided Adaptive Detector Operating-Point Control for Aerial Multi-Object Tracking

Ahmed Gouda Ismail^{1*}, Mohamed S. Mohamed¹ and Tarek Ahmed Mahmoud²

^{1*}Computer Engineering and Artificial Intelligence Department, Military Technical College, Cairo, Egypt.

²Computer Engineering Department, Egypt University of Informatics, Cairo, Egypt.

*Corresponding author(s). E-mail(s): a7medgouda1@gmail.com;

Contributing authors: mohamedms@mtc.edu.eg; tarek.mohamed@eui.edu.eg;

Abstract

Scenes recorded by unmanned aerial vehicles change over time in object density, object size, illumination, and blur. Most tracking-by-detection pipelines still run the detector at one fixed operating point. AC-MOT is a training-free control layer that treats the detector operating point as part of the multi-object tracking (MOT) system. It uses only the current frame and past tracker output. At run time it reads inexpensive scene cues and selects the detector confidence threshold, the non-maximum-suppression (NMS) IoU threshold, and the input resolution. The detector and the tracker are not retrained or modified. The controller was developed through optimization: a quality-oriented Optuna profile, an identity-aware MOTA/IDS Pareto study, transfer and cross-pipeline studies, and a modern controller developed with sequence-level cross-validation on separate calibration sequences and then frozen. On a frozen evaluation of seven VisDrone2019-MOT sequences with a YOLO11m detector, AC-MOT raises HOTA by 2.937 points with ByteTrack (95% CI [2.356, 3.375]) and by 1.325 points with the recent OATrack tracker (95% CI [0.200, 2.264]); MOTA rises by 17.995 and 6.318 points. On a Tesla T4 GPU, the controller adds only 4.26 ms per frame with ByteTrack and 4.20 ms with OATrack, so it adds very little latency and is suitable for real-time use. Applied to UAVDT without retuning, AC-MOT raises MOTA by 17.601 points with ByteTrack and by 3.267 points with OATrack.

Keywords: multi-object tracking, UAV video, adaptive inference, detector operating point, real-time image processing

1 Introduction

Unmanned aerial vehicles (UAVs) are widely used for traffic monitoring, surveillance, and search, and many of these tasks need multi-object tracking (MOT): every object must be detected in every frame and keep the same identity over time [1, 2]. Aerial video makes this hard. Objects are small, their number changes quickly, the camera moves, and illumination and blur change within

one video [2–4]. Many of these applications also need results with low delay, so the processing cost matters as much as the accuracy.

Most MOT systems follow the tracking-by-detection design. A detector finds the objects in each frame, and a tracker links the detections over time [5, 6]. Much research improves the tracker, for example its motion model or its association rules, and much research improves the detector

itself. Far less attention is given to how the detector is run. In practice the detector uses one confidence threshold, one non-maximum-suppression (NMS) threshold, and one input resolution for the whole video. These three values form the detector operating point, and they decide which boxes reach the tracker.

A fixed operating point does not fit a scene that keeps changing. A setting that works in a sparse, clear scene can keep too much clutter in a dense scene. A setting that works in a dense scene can lose small or weak objects in another part of the same video. A large input helps small objects, but it costs the same time on easy and hard frames. The choice therefore moves false positives, missed objects, identity switches, and runtime together.

Existing adaptive methods only partly address this problem. Adaptive inference methods change the detector’s input scale or configuration, but they usually need an extra learned model and are designed for detection, not tracking [7, 8]. Adaptive tracking methods change thresholds inside one specific tracker [9, 10]. What is missing is a simple, training-free way to adapt the detector operating point online for MOT, without changing the detector or the tracker.

This paper presents AC-MOT, a training-free control layer that adapts the detector operating point online for aerial MOT. It sits between the detector and the tracker, reads inexpensive scene cues, and works with existing detectors and trackers without retraining or modifying them. The method is described in Sect. 3, and Sect. 4 explains how it was developed with optimization and tested step by step.

The main contributions are:

1. A training-free control layer that treats the detector operating point as part of the MOT system and adapts it online (Sect. 3).
2. An optimization-guided development path, in which each step is compared with the baseline it was built on (Sect. 4).
3. An evaluation on VisDrone2019-MOT with two trackers, ByteTrack and the recent OATrack, using eight MOT metrics and paired bootstrap intervals, together with a transfer to UAVDT without retuning (Sects. 5 and 6).

4. A latency measurement showing that the control layer adds only a few milliseconds per frame on a Tesla T4 GPU (Sect. 7).

The rest of the paper is organized as follows. Section 2 reviews related work. Section 3 describes AC-MOT, and Section 4 explains how it was developed. Section 5 gives the experimental setup, Section 6 the results, and Section 7 a discussion. Section 8 concludes the paper.

2 Related Work

2.1 Tracking by detection and aerial MOT

SORT built a compact online pipeline from motion prediction and assignment, and DeepSORT added appearance features to reduce identity fragmentation [5, 11]. ByteTrack first associates high-confidence detections and then uses low-confidence boxes to recover objects that would otherwise be lost [6]. Tracktor reuses the detector’s box regression for tracking [12], and TrackFormer performs detection and association with transformer queries [13]. Standard evaluation follows the MOTChallenge benchmarks [14].

Aerial video adds small targets, changing object counts, camera motion, and clutter [2, 3]. UAVMOT addresses the motion of the UAV camera inside the tracker [15], and LSO-YOLO designs a lightweight real-time detector for small UAV objects [16]. OATrack [17] is a recent UAV small-object tracker published in 2026. It weights association by detection quality and uses a YOLO detection cache. We use OATrack as a second host and keep it unchanged. With the same detector, AC-MOT improves its results in our pipeline: HOTA rises by 1.325, MOTA by 6.318, and IDF1 by 2.258 points, and IDS fall by 169 (Sect. 6). These works improve the tracker or the detector itself. AC-MOT asks a different question: which detector setting should be used for the current scene.

2.2 Adaptive thresholds inside trackers

Some trackers adapt their own thresholds. Adaptive ByteTrack changes the confidence threshold used inside ByteTrack [9], and AdapTrack adapts the matching thresholds of the association step

Table 1 Adaptive methods closest to AC-MOT (BT: ByteTrack).

Method	Adapted	Where	Task
AdaScale [7]	input scale	detector	video detection
ApproxDet [8]	detection configuration	detector	video detection
Chameleon [20]	pipeline configuration	analytics pipeline	video analytics
Adapt. BT [9]	confidence threshold	inside BT	MOT
AdapTrack [10]	matching thresholds	inside the tracker	MOT
AC-MOT	confidence, NMS, resolution	detector; tracker unchanged	aerial MOT

[10]. Both change one tracker from the inside. AC-MOT works outside the tracker: it adapts the detector setting and leaves the tracker unchanged, so the same controller runs with ByteTrack and with OATrack without any change to either. It also adapts three settings together (confidence, NMS, and resolution) instead of one threshold.

2.3 Adaptive inference for detection

Dynamic neural networks change their structure or parameters for each input [18], and resolution-adaptive networks spend less computation on easy inputs [19]. AdaScale selects the input scale of each video frame with a scale regressor trained on detector features [7]. ApproxDet switches detection configurations at run time with accuracy and latency models trained offline [8]. Chameleon re-profiles the configuration of video analytics pipelines over time [20]. These methods target detection accuracy or analytics cost, and most of them need an extra learned model. AC-MOT needs no learned model and no retraining, and it is evaluated with MOT metrics on two trackers. Table 1 summarizes the differences.

2.4 Optimization and evaluation

Optuna provides define-by-run search with the Tree-structured Parzen Estimator (TPE) sampler [21], which was used throughout the development of AC-MOT. The identity-aware study reports a feasible Pareto front based on the usual notion of non-dominance [22]. MOTA weighs detection and identity errors, IDF1 measures identity precision and recall, and HOTA balances detection and association [23]. Paired bootstrap resampling

at the sequence level keeps the dependence within each sequence and gives intervals for system differences [24].

3 AC-MOT Method

3.1 Overview and operating point

Figure 1 shows the controlled pipeline. Each frame passes through scene analysis, the AC-MOT controller, the detector, and the tracker. The controller chooses the detector operating point

$$\theta_t = (c_t, \eta_t, r_t), \quad (1)$$

where c_t is the detector confidence threshold, η_t is the NMS IoU threshold, and r_t is the input resolution (the long side of the resized image, in pixels). The detector resizes the frame to r_t , suppresses boxes that overlap a higher-scoring box by more than η_t , and removes boxes with a score below c_t . The remaining boxes go to the tracker. The detector weights and the tracker are frozen; AC-MOT only changes θ_t .

The controller uses only information that is available at run time. Its decision for frame t is

$$\theta_t = \pi(I_t, \mathcal{B}_{1:t-1}), \quad (2)$$

where I_t is the current image and $\mathcal{B}_{1:t-1}$ are the boxes that the tracker reported for earlier frames. The current image is available when the decision is made, so its statistics may be used. Tracker output of frame t is stored only after detection and association, and it is used from frame $t+1$ on. No ground truth and no future frame enter the decision. The scene layer reads box coordinates and image statistics only; detector scores do not enter it.

3.2 Scene analysis

Scene analysis runs on frame 1 and then every ten frames ($t = 1$ or $t \bmod 10 = 1$); analysis step j denotes the j -th such frame. Two kinds of cues are computed.

Image cues of the current frame. The frame is reduced to quarter scale and converted to gray. The edge cue e_t is the fraction of Canny edge pixels (thresholds 50 and 120), the brightness cue β_t is the mean gray level, and the blur cue ν_t is the variance of the Laplacian.

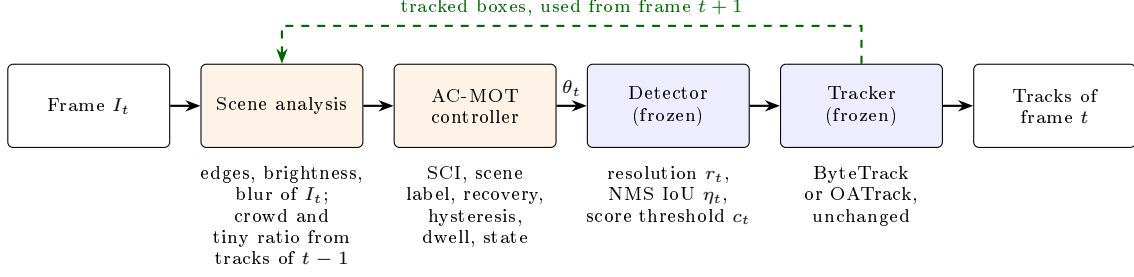


Fig. 1 AC-MOT in a tracking-by-detection pipeline. Scene analysis and the controller choose the operating point $\theta_t = (c_t, \eta_t, r_t)$ before the detector runs. The frozen detector and tracker are not modified. Tracked boxes of frame t feed the scene analysis of later frames (dashed arrow).

Box cues of earlier frames. Let N_j be the number of boxes that the tracker reported for frame $t-1$. The crowd cue and the tiny-object ratio are

$$C_j = \min\left(\frac{N_j}{30}, 1\right), \quad (3)$$

$$T_j = \frac{1}{N_j} \sum_{i=1}^{N_j} \mathbf{1}[a_i < 32^2], \quad (4)$$

where a_i is the area of box i in image pixels and $T_j = 0$ when $N_j = 0$.

3.3 Crowd-selected SCI and scene label

The SCI of the frozen controller is the mean of the most recent crowd values,

$$S_j = \frac{1}{|H_j|} \sum_{k \in H_j} C_k, \quad (5)$$

where H_j holds the last seven analysis steps (fewer at the start of a sequence). The weighted SCI uses the crowd cue alone because the cue audit of Sect. 4.4 selected the subset {crowd}. The other cues are still computed and still act on the state, as guard and auxiliary cues. They define a scene label ℓ_j : *night* if $\beta_t < 80$; otherwise *blur* if $\nu_t < 180$; otherwise *tiny* if $T_j > 0.50$; otherwise *crowded* if $C_j > 0.65$ or $e_t > 0.13$; otherwise *clear*. The tiny ratio also guards the HIGH state directly, as shown next.

3.4 Three-state controller

The controller has three states, LOW, MEDIUM, and HIGH. The names describe scene difficulty. They are not a ranking of input resolution: in the

frozen action map the HIGH state uses the smallest input (Table 3). At each analysis step the state is updated in the order of Fig. 2 and Algorithm 1.

Target. The target is HIGH if $S_j > \tau_h$ or $T_j > 0.50$, MEDIUM if $S_j > \tau_m$ or $\ell_j \in \{\text{crowded, tiny}\}$, and LOW otherwise, with the frozen thresholds τ_m and τ_h of Table 2.

Recovery probe. The controller keeps a decaying peak P_j of the box count. When $P_{j-1} \geq 5$ and $N_j < 0.40 P_{j-1}$, a drop age d_j is increased; otherwise $d_j = 0$. The peak is then updated as $P_j = \max(N_j, \lfloor 0.8 P_{j-1} \rfloor)$. While $1 \leq d_j \leq 3$, the target is set to HIGH. This gives a short, bounded response when the number of tracked objects collapses.

Hysteresis. Only downward moves are held. In the HIGH state, a lower target is ignored while $S_j > 0.50$ or $T_j > 0.40$. In the MEDIUM state, a lower target is ignored while $S_j > 0.25$.

Dwell. The state cannot change within 30 frames of the last change. The controller starts in LOW, and the state is held between analysis frames.

3.5 Frozen constants and action map

Table 2 lists the frozen constants and Table 3 the frozen action map. LOW uses the largest input (1536 pixels) with loose NMS and a low score threshold. MEDIUM uses 1280 pixels with tight NMS and a score threshold of 0.40. HIGH uses 1088 pixels with NMS 0.60 and a score threshold of 0.40. The actions were selected on calibration data together with τ_m and τ_h (Sect. 4.4). The score threshold c_t is applied on the detector side before the tracker. It is separate from any confidence gate inside the tracker, such as the fixed tracker-side gate of OATrack.

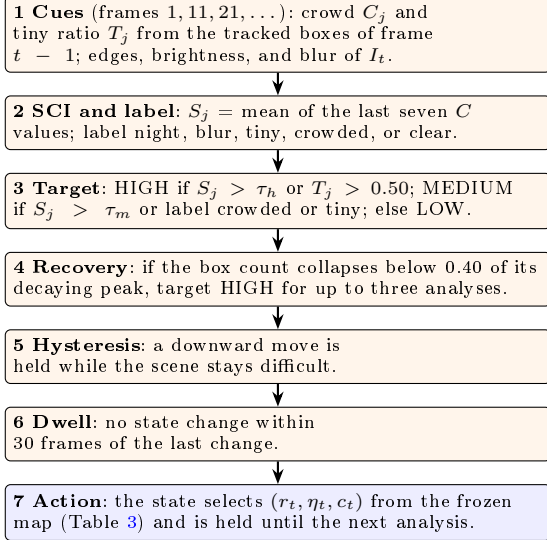


Fig. 2 Update order of the frozen three-state controller at an analysis frame.

3.6 How the constants were set

The constants were set with parameter sweeps and experiments, not by hand. The analysis stride and the SCI window come from a temporal sweep on the VisDrone validation sequences. It tested 25 combinations, with windows of 1, 3, 5, 7, and 9 analyses and strides of 1, 5, 10, 15, and 20 frames. Among the settings that met the speed and identity constraints (at least 25 FPS and at most 271 IDS), a window of 7 and a stride of 10 gave the highest MOTA. Operating-point sweeps on the same data covered input resolutions from 512 to 960 pixels, confidence thresholds from 0.05 to 0.50, and NMS thresholds from 0.30 to 0.80, and defined the supported values of the first optimized profiles (Sect. 4). The cue subset, the thresholds τ_m and τ_h , and the action map were selected by cross-validated search on separate calibration sequences (Sect. 4.4). The remaining constants of Table 2 were kept fixed in all development stages.

4 How the Final Controller Was Developed

The controller was developed in four steps: a quality-oriented profile, an identity-aware profile, transfer and cross-pipeline studies, and a modern stage that produced the frozen controller. Each step is compared with the baseline it was built on.

Algorithm 1 Frozen AC-MOT controller for one sequence

```

1:  $q \leftarrow \text{LOW}$ ;  $H \leftarrow \emptyset$ ;  $P, d \leftarrow 0$ ;  $t_c \leftarrow 1$ 
2: for each frame  $t = 1, 2, \dots$  do
3:   if  $t = 1$  or  $t \bmod 10 = 1$  then
4:      $e_t, \beta_t, \nu_t \leftarrow$  statistics of  $I_t$ 
5:      $N, C, T \leftarrow$  box cues of frame  $t - 1$ 
6:     add  $C$  to  $H$  (keep 7);  $S \leftarrow \text{mean}(H)$ 
7:      $\ell \leftarrow$  scene label;  $g \leftarrow$  target from  $S, T, \ell$ 
8:     if  $P \geq 5$  and  $N < 0.4P$  then  $d \leftarrow d + 1$ 
9:     else  $d \leftarrow 0$ 
10:    end if
11:     $P \leftarrow \max(N, \lfloor 0.8P \rfloor)$ 
12:    if  $1 \leq d \leq 3$  then  $g \leftarrow \text{HIGH}$ 
13:    end if
14:    apply the downward holds of Table 2
15:    if  $t - t_c \geq 30$  and  $g \neq q$  then
16:       $q \leftarrow g$ ;  $t_c \leftarrow t$ 
17:    end if
18:  end if
19:   $(r_t, \eta_t, c_t) \leftarrow$  action of state  $q$ 
20:  detect with  $(r_t, \eta_t)$ ; drop scores  $< c_t$ 
21:  update the tracker; store its boxes
22: end for

```

Table 2 Frozen constants of the AC-MOT scene layer.

Constant	Value
Analysis stride (frames)	10
SCI window (analysis steps)	7
Crowd normalizer (boxes)	30
Tiny-object area (pixels)	32×32
MEDIUM threshold τ_m	0.29747709701135766
HIGH threshold τ_h	0.5114928923560124
Tiny guard for HIGH	0.50
Label thresholds: brightness, blur	80, 180
Label thresholds: crowd, edges	0.65, 0.13
Recovery: min. peak, drop ratio, peak decay, steps	5, 0.40, 0.80, 3
Holds: HIGH (S, T), MEDIUM (S)	(0.50, 0.40), 0.25
Dwell (frames)	30
Initial state	LOW

The historical steps used other detectors and evaluation protocols than the modern stage, so their numbers are reported in their own tables and are not pooled with the modern results.

4.1 Quality-oriented profile

Baseline. The reference pipeline is the pretrained YOLOv8n detector [25] followed by ByteTrack [6], both at their default settings: confidence threshold 0.25, NMS IoU threshold 0.45, input size

Table 3 Frozen action map. State names describe scene difficulty, not a resolution rank.

State	Resolution r	NMS IoU η	Score threshold c
LOW	1536	0.70	0.10
MEDIUM	1280	0.45	0.40
HIGH	1088	0.60	0.40

640 pixels, and the default ByteTrack parameters (high threshold 0.25, low threshold 0.10, new-track threshold 0.25, buffer 30 frames, match threshold 0.80). It runs with one fixed operating point for the whole video.

Method. The first optimization-derived profile kept the same detector, used a tuned ByteTrack profile, and searched only on the VisDrone2019-MOT validation split. Five cues were used: crowd count, the fraction of earlier boxes below 32×32 pixels, edge density, low brightness, and low Laplacian variance. Each cue was normalized with validation statistics, and the index was their weighted sum

$$S_j = \sum_{k \in \{c, t, e, n, b\}} w_k x_{k,j}, \quad (6)$$

with $w_k \geq 0$ and $\sum_k w_k = 1$, averaged over seven analyses taken every ten frames. Confidence and NMS were interpolated linearly between the values at $S = 0$ and $S = 1$, and two sorted thresholds selected one of three resolutions (512, 928, 960 pixels).

An Optuna TPE study [21] jointly searched the five weights, the confidence and NMS end points, and the two thresholds. The resolution levels, the supported confidence values $\{0.25, \dots, 0.45\}$, the supported NMS values from 0.30 to 0.70, and the temporal settings were fixed beforehand by validation sweeps (Sect. 3.6). The selection rule kept trials with at least 25 FPS and at most 271 IDS (a reference value from earlier validation runs) and then took the highest MOTA, with lower IDS, HOTA, IDF1, and FPS as tie-breaks. The frozen profile (archived as Trial 24) has $w_c = 0.1294928$, $w_t = 0.2217477$, $w_e = 0.4337134$, $w_n = 0.0535577$, $w_b = 0.1614885$, confidence end points (0.30, 0.40), NMS end points (0.35, 0.35), and thresholds (0.1353494, 0.2872868). On validation

Table 4 Quality-oriented profile versus its baseline (YOLOv8n with ByteTrack at default settings) on VisDrone test-dev, historical protocol. Gain: paired bootstrap difference with 95% interval; for IDS it is the reduction.

Metric	Baseline	Quality	Gain [95% CI]
HOTA	28.430	33.835	+5.4051 [4.1273, 6.9022]
MOTA	19.729	26.948	+7.2195 [5.4362, 9.2934]
IDF1	32.724	41.546	+8.8220 [6.9005, 11.0537]
IDS	1235	1184	51 [-114, 219]
FPS	36.528	38.985	-

it reached MOTA 23.038, HOTA 36.110, IDF1 40.758, and IDS 270 at 37.169 FPS.

Comparison. Table 4 compares the profile with the baseline on the historical VisDrone test-dev split, with paired bootstrap intervals (5000 resamples, seed 42). HOTA rises by 5.4051 points, MOTA by 7.2195 points, and IDF1 by 8.8220 points, and throughput rises from 36.528 to 38.985 FPS.

4.2 Identity-aware Pareto profile

Baseline. The baseline is the same YOLOv8n and ByteTrack pipeline at default settings (Sect. 4.1).

Method. Quality is not the only deployment goal; identity stability and compute can move in a different direction from MOTA. The second study kept the same cues, operating levels, temporal settings, and validation data, but changed the question. It maximized MOTA and minimized IDS subject only to $\text{FPS} \geq 25$, without the IDS gate. A seeded TPE sampler ran 49 completed trials. A trial dominated another when it had no lower MOTA and no higher IDS, with at least one strict improvement, and the feasible Pareto front was built from these relations [22]. The balanced candidate maximized an equal-weight score of normalized MOTA and normalized IDS reduction, with HOTA, IDF1, FPS, and trial index as tie-breaks.

The selected profile (archived as Trial 22) has weights $w_c = 0.1646453$, $w_t = 0.1746265$, $w_e = 0.5076113$, $w_n = 0.1206956$, $w_b = 0.0324213$, confidence end points (0.40, 0.40), NMS end points (0.60, 0.35), and thresholds (0.2927136, 0.6661672). On validation it reached MOTA 19.330, HOTA 31.651, IDF1 34.226, and IDS 168 at 51.406 FPS.

Table 5 Identity-aware profile versus its baseline (YOLOv8n with ByteTrack at default settings) on VisDrone test-dev, historical protocol. Gain: paired bootstrap difference with 95% interval; for IDS it is the reduction.

Metric	Baseline	Identity	Gain [95% CI]
HOTA	28.430	31.218	+2.788 [0.747, 4.660]
MOTA	19.729	23.792	+4.063 [0.952, 6.886]
IDF1	32.724	37.870	+5.146 [2.028, 8.080]
IDS	1235	919	316 [175, 470]
FPS	36.528	46.024	–

Table 6 Historical UAVDT transfer without retuning. Baseline: YOLOv8n with ByteTrack at default settings.

Metric	Baseline	Quality	Identity
HOTA	24.085	28.390	26.930
MOTA	13.841	17.399	16.118
IDF1	27.887	34.453	32.014
IDS	558	321	308
FPS	65.015	58.353	61.462

Comparison. Table 5 compares the profile with the baseline on test-dev. It removes 316 identity switches and raises throughput from 36.528 to 46.024 FPS, while HOTA, MOTA, and IDF1 also rise. Against the quality-oriented profile it has 265 fewer IDS [82, 502] and -2.617 HOTA $[-4.004, -1.694]$. The two profiles therefore answer different questions: one favors overall quality and the other favors identity stability and speed.

4.3 Transfer and cross-pipeline studies

UAVDT. The baseline is again YOLOv8n with ByteTrack at default settings (Sect. 4.1). Both frozen profiles were applied to UAVDT [3] without retuning (Table 6). The quality-oriented profile raised HOTA by 4.305 points [2.740, 5.709] and the identity-aware profile by 2.845 points [1.238, 4.348]; their IDS reductions are 237 [104, 377] and 250 [110, 404].

U2MOT on VisDrone. The baseline is the published U2MOT tracker [26] with its released YOLOX-X detector [27], run at its own static setting. Seven VisDrone validation sequences served for development and 17 test-dev sequences for evaluation. The cue weights and transforms of the

Table 7 Cross-pipeline study on VisDrone test-dev. Baseline: U2MOT with its YOLOX-X detector at its static setting.

Metric	U2MOT baseline	With controller
MOTA	53.873	53.767
IDF1	69.775	69.849
IDS	1239	1152
FP	41385	40155
FN	63241	64801

Table 8 Cross-pipeline study on MOT17 val_half. Baseline: SparseTrack with a fixed NMS IoU threshold of 0.75.

Metric	SparseTrack baseline	With adaptive NMS
MOTA	76.8881	76.9252
IDF1	81.4943	81.5299
IDS	136	123

quality-oriented profile were kept, the tracker was not modified, and the action boundaries were set to the 33rd and 67th percentiles of the index on the U2MOT validation run, without using ground truth. Table 7 gives the comparison: the controlled pipeline has 87 fewer IDS and 1230 fewer FP.

SparseTrack on MOT17. The baseline is the published SparseTrack tracker [28] with a fixed NMS IoU threshold of 0.75, on the seven MOT17 val_half sequences (2652 frames) [14]. A minimal edge-driven rule switched only the NMS threshold, between 0.70 and 0.80; its threshold came from a leave-one-sequence-out study on the same sequences. Table 8 gives the comparison: the adaptive rule reduces IDS from 136 to 123, with MOTA +0.0371 points $[-0.041, +0.182]$.

Together these studies show that the preferred profile depends on the objective (quality or identity) and on the tracker host. This motivated a modern stage with a stronger detector, explicit calibration data, and two tracker hosts.

4.4 Modern development of the frozen controller

Baseline. The modern baseline is the YOLO11m detector [29], in the public VisDrone checkpoint described in Sect. 5, run at a fixed operating point of 1536 pixels and NMS IoU 0.70 with no extra score threshold. It feeds ByteTrack [6] during development and both ByteTrack and OATrack

[17] in the final evaluation. A large fixed input is a common choice for small aerial objects when no adaptation is used.

Data and protocol. Eight sequences of the VisDrone2019-MOT training split (3282 frames) were declared as calibration data. They are disjoint from the seven evaluation sequences. All development used four folds, each holding out two calibration sequences, and reported the cross-validated (CV) mean change in HOTA against the baseline. Detector outputs were cached for every resolution and NMS value; a score threshold applied to the cache was checked to reproduce direct inference exactly. Table 9 summarizes the development steps.

Cue audit. Stages 1 and 2 searched the state-to-resolution and state-to-NMS maps on exhaustive grids. Stage 3A searched all 31 non-empty subsets of the five cues and selected {crowd}, which defines the crowd-selected SCI of Eq. (5). The other cues were kept as guard and auxiliary cues of the state logic (Sect. 3).

Score threshold. At 1088 pixels and NMS 0.45, a fixed score threshold of 0.40 raised pooled calibration HOTA by +0.881 over 0.01 and improved seven of the eight sequences. The score threshold was therefore added as the third action dimension.

Final joint search. The final search used 60 TPE trials (seed 43). It searched $\tau_m \in [0.10, 0.50]$, $\tau_h \in [\tau_m + 0.05, \min(\tau_m + 0.40, 0.90)]$, and for each state a resolution in {1088, 1280, 1536}, an NMS value in {0.45, 0.60, 0.70}, and a score threshold in {0.10, 0.25, 0.40}. The objective was the CV mean change in HOTA against the baseline, with ByteTrack as host. Eligible candidates had to realize at least two distinct actions, each on at least 10% of calibration frames, and switch within sequences. The frozen policy is the eligible candidate whose three states use three different actions (Table 3). On calibration data it used LOW, MEDIUM, and HIGH on 7.6%, 10.1%, and 82.3% of frames, with 0.607 state changes per 100 frames on average.

Comparison. Table 10 compares the frozen policy with the baseline on each calibration fold. All four folds are positive, with a CV mean of +2.447 HOTA. After these steps the thresholds, cue subset, and action map were frozen. The final comparison on the evaluation sequences is given in Sect. 6.

Table 9 Main modern development steps on the eight calibration sequences (ByteTrack host). The score-threshold gain is pooled HOTA; the joint-search result is the mean over four folds.

Step	Search	Result
Cue subset	31 subsets	{crowd}
Score threshold	4 values	0.40; +0.881 HOTA
Joint search	60 TPE trials	CV mean +2.447

Table 10 Frozen policy versus the modern baseline (YOLO11m at 1536 pixels and NMS 0.70, ByteTrack) on the four calibration folds: change in HOTA.

	Fold 1	Fold 2	Fold 3	Fold 4	Mean
Δ HOTA	+0.592	+4.164	+2.029	+3.004	+2.447

5 Experimental Setup

5.1 Detector and tracker hosts

All modern experiments use one frozen public YOLO11m detector [29], the Hugging Face checkpoint `dronefreak/visdrone-yolo11m`. It was trained on VisDrone2019-DET at 640 pixels (SHA256 `c6f98247e7c731a567aaf37ca7b808beff588b687623e028b25a2e1d846e96e7`). It runs in half precision. Detections of the classes pedestrian, car, van, truck, and bus are passed on.

Two tracker hosts are used and kept unchanged. ByteTrack uses the Ultralytics implementation with high threshold 0.25, low threshold 0.10, new-track threshold 0.25, buffer 30 frames, and match threshold 0.80. OATrack is a re-implementation of the published method [17] in its main configuration, with a tracker-side confidence gate of 0.40. This gate belongs to the tracker and is never changed by AC-MOT.

5.2 Systems

The baseline runs the detector at the fixed operating point of 1536 pixels and NMS IoU 0.70, with no score threshold above the cache floor of 0.01. AC-MOT runs the frozen controller of Sect. 3 on the same detector. For each host, the baseline and AC-MOT differ only in the detector operating point.

5.3 Data

The VisDrone evaluation uses seven sequences of the VisDrone2019-MOT validation split. They contain 71,830 ground-truth boxes. The cue subset, thresholds, and action map of the frozen controller were selected on the separate calibration sequences only (Sect. 4.4). The UAVDT transfer uses 20 test sequences with 16,592 frames. The frozen controller, thresholds, and action map are applied without any change. Detector outputs for UAVDT were produced at the three frozen operating points, giving 49,776 detector forward passes.

5.4 Metrics and evaluation

VisDrone results are scored with a Python port of the VisDrone2019-MOT Task 4b evaluation. Ignored regions are removed, the five classes above are evaluated separately, CLEAR-MOT and identity measures use IoU 0.5, and counts are summed over classes and sequences. HOTA is added with the same class-aware scheme using TrackEval. UAVDT is scored with one vehicle class. We report HOTA [23], MOTA, IDF1, IDS, FP, FN, precision, and recall.

Uncertainty on VisDrone is estimated with a paired bootstrap at the sequence level [24]. Each of 5000 resamples (seed 42) draws seven sequences with replacement. The same draw is used for both systems, the pooled metrics are recomputed, and their difference is recorded. The 95% interval is the 2.5–97.5 percentile range. UAVDT is a single frozen run per host.

Latency is measured on one Tesla T4 GPU over the first 200 frames of sequence `uav0000086_00000_v`, excluding five warm-up frames. For every frame, the time spent in the controller (image statistics, scene analysis, and state update) is recorded separately from the detector and tracker time.

6 Results

6.1 Frozen VisDrone evaluation

Table 11 lists the aggregate results on the seven VisDrone sequences, and Fig. 3 shows HOTA, MOTA, and IDF1.

ByteTrack host. AC-MOT changes HOTA from 41.586 to 44.523, MOTA from 9.876 to

Table 11 Frozen VisDrone evaluation on seven VisDrone2019-MOT sequences (71,830 ground-truth boxes). Baseline: fixed operating point (1536, 0.70).

Metric	ByteTrack		OATrack	
	Baseline	AC-MOT	Baseline	AC-MOT
HOTA	41.586	44.523	46.770	48.095
MOTA	9.876	27.871	28.165	34.483
IDF1	48.569	53.929	57.107	59.364
IDS	1341	1016	773	604
FP	39274	24892	25394	21400
FN	24121	25902	25432	25057
Precision	54.849	64.852	64.628	68.609
Recall	66.419	63.940	64.594	65.116

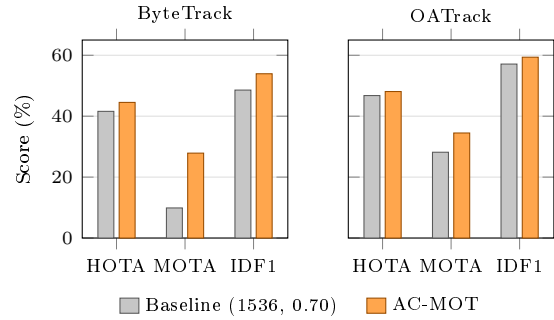


Fig. 3 HOTA, MOTA, and IDF1 on the seven VisDrone sequences for both tracker hosts (values in Table 11).

27.871, and IDF1 from 48.569 to 53.929. IDS fall from 1341 to 1016 and FP from 39274 to 24892, while FN rise from 24121 to 25902. Precision rises from 54.849 to 64.852 and recall falls from 66.419 to 63.940. The detector output becomes more selective: clutter and identity-confusing boxes are removed at a small cost in recall.

OATrack host. AC-MOT changes HOTA from 46.770 to 48.095, MOTA from 28.165 to 34.483, and IDF1 from 57.107 to 59.364. IDS fall from 773 to 604, FP from 25394 to 21400, and FN from 25432 to 25057. Precision rises from 64.628 to 68.609 and recall from 64.594 to 65.116. OATrack starts from a stronger baseline.

6.2 Paired bootstrap

Table 12 gives the paired differences (AC-MOT minus baseline) and their 95% intervals. With ByteTrack, the intervals of HOTA, MOTA, IDF1, precision, IDS, and FP lie entirely on the side of improvement, and FN and recall move in the opposite direction. With OATrack, HOTA,

Table 12 Paired sequence-level bootstrap on VisDrone: AC-MOT minus baseline, 5000 resamples, seed 42. HOTA, MOTA, IDF1, precision, and recall in points; IDS, FP, and FN in counts.

Host	Metric	Difference	95% interval
ByteTrack	HOTA	+2.937	[+2.356, +3.375]
	MOTA	+17.995	[+13.541, +23.282]
	IDF1	+5.360	[+4.490, +6.178]
	IDS	-325	[-498, -164]
	FP	-14382	[-19176, -9620]
	FN	+1781	[+288, +3594.075]
	Precision	+10.003	[+8.712, +11.895]
	Recall	-2.479	[-4.507, -0.469]
OATrack	HOTA	+1.325	[+0.200, +2.264]
	MOTA	+6.318	[+4.741, +7.879]
	IDF1	+2.258	[+0.548, +3.474]
	IDS	-169	[-292, -42]
	FP	-3994	[-6190, -2494.900]
	FN	-375	[-1462.575, +670]
	Precision	+3.981	[+2.957, +5.511]
	Recall	+0.522	[-0.869, +2.111]

Table 13 Frozen UAVDT transfer without retuning (20 sequences, 16,592 frames, one vehicle class). Baseline: fixed operating point (1536, 0.70).

Metric	ByteTrack		OATrack	
	Baseline	AC-MOT	Baseline	AC-MOT
HOTA	45.923	47.004	47.728	47.640
MOTA	-1.573	16.028	19.607	22.873
IDF1	58.400	61.767	62.101	62.931
IDS	601	486	517	486
FP	279616	212077	197937	183669
FN	66050	73702	75612	78774
Precision	49.571	55.751	57.270	58.800
Recall	80.625	78.381	77.820	76.893

MOTA, IDF1, and precision rise and IDS and FP fall, all with intervals on one side of zero.

6.3 Controller behaviour

The controller uses all three states with both hosts. With ByteTrack, LOW, MEDIUM, and HIGH cover 7.4%, 30.1%, and 62.5% of the frames; with OATrack they cover 7.4%, 33.3%, and 59.3%. The two hosts diverge through their different tracked boxes. State changes are rare, between 0.10 and 0.54 per 100 frames depending on the sequence, so the detector input stays stable over long stretches of video.

6.4 Frozen UAVDT transfer

Table 13 and Fig. 4 report the transfer to UAVDT without retuning. With ByteTrack,

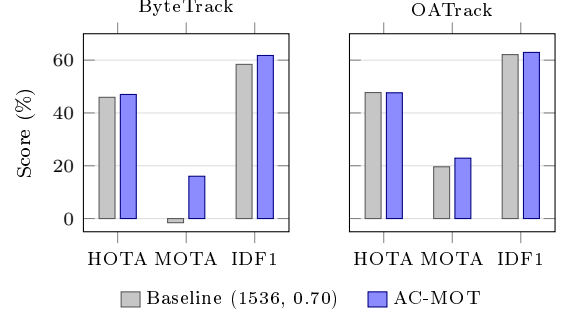


Fig. 4 HOTA, MOTA, and IDF1 of the frozen UAVDT transfer for both tracker hosts (values in Table 13).

HOTA changes from 45.923 to 47.004 (+1.081), MOTA from -1.573 to 16.028 (+17.601), and IDF1 from 58.400 to 61.767 (+3.367). IDS fall by 115 and FP by 67539, while FN rise by 7652. With OATrack, MOTA changes from 19.607 to 22.873 (+3.267), IDF1 from 62.101 to 62.931 (+0.831), IDS fall by 31, and FP fall by 14268. With both hosts the transfer gives fewer FP and IDS and higher precision, with slightly lower recall, the same pattern as with ByteTrack on VisDrone.

7 Discussion

The operating point is part of the MOT system. The same frozen detector and the same trackers give very different MOT results at different operating points. Moving away from the fixed 1536/0.70 point raises MOTA by 17.995 points with ByteTrack and by 6.318 points with OATrack on VisDrone. The detector operating point is therefore a first-order variable of aerial MOT and deserves the same attention as the association method.

Selective observation streams. The main effect of AC-MOT is a more selective detection stream. FP and IDS drop sharply, precision rises, and FN rise a little with ByteTrack. MOTA reacts most strongly because it counts FP directly; HOTA and IDF1 rise by smaller amounts. The same pattern appears on UAVDT, where FP fall by 67539 with ByteTrack and by 14268 with OATrack.

Tracker dependence. ByteTrack gains more than OATrack. ByteTrack starts from a weaker baseline and uses low-score boxes in its second association step, so a likely reason is that removing clutter helps it directly. OATrack already

weights detections by quality inside its association, and it gains mainly in MOTA, IDF1, IDS, and FP. Results are therefore reported per host.

Development path. The historical profiles showed that the preferred operating point depends on the goal: quality or identity stability and speed. The modern stage kept a finite, readable controller with one crowd index, a few guard cues, three states, and three actions.

Deployment. AC-MOT needs no training and no access to detector or tracker internals. It changes three arguments of the detector call. Its analysis runs on a quarter-scale image every ten frames and uses boxes that the tracker already produced. On a Tesla T4 the controller takes 4.26 ms per frame with ByteTrack and 4.20 ms with OATrack, which is small next to detector inference. The control layer therefore adds very little latency and is suitable for real-time use. The same interface can host other action maps, for example maps chosen for a latency budget, without changes to the detector or the tracker.

8 Conclusion

AC-MOT is a training-free control layer that uses only the current frame and past tracker output and adapts the detector confidence threshold, NMS IoU threshold, and input resolution online while the detector and the tracker stay unchanged. Its frozen form uses a crowd-selected scene complexity index, guard cues for tiny objects, illumination, blur, and edges, a recovery probe, hysteresis, and dwell, and a three-state action map.

The controller was developed through optimization. Quality-oriented and identity-aware profiles, together with transfer and cross-pipeline studies, showed that the preferred operating point depends on the objective and on the tracker host. The modern stage froze the controller after sequence-level cross-validation on separate calibration sequences.

On seven VisDrone2019-MOT sequences, AC-MOT raised HOTA by 2.937 points with ByteTrack and by 1.325 points with OATrack, and MOTA by 17.995 and 6.318 points, with paired bootstrap intervals for every metric. Applied to UAVDT without retuning, it raised MOTA by 17.601 and 3.267 points. These results support

treating the detector operating point as part of the MOT system and controlling it online.

Declarations

Funding. This research received no external funding.

Conflict of interest. The authors declare no competing interests.

Ethics approval and consent to participate. This computational study uses public benchmark data and does not involve human participants or animals.

Consent for publication. Not applicable.

Data and code availability. The frozen controller specification, result records, evidence map, staged sections, scripts, and build package are included in the project repository. Dataset redistribution remains subject to the original benchmark licenses and access terms.

Author contributions. Ahmed Gouda Ismail: Conceptualization, Methodology, Software, Investigation, Data curation, Formal analysis, Visualization, Writing – original draft, Writing – review & editing. Mohamed S. Mohamed: Supervision, Methodology, Writing – review & editing. Tarek Ahmed Mahmoud: Supervision, Validation, Writing – review & editing.

Acknowledgements. Not applicable.

Submission exclusivity. The manuscript is not under consideration elsewhere and has not been published previously.

AI use in manuscript preparation. Generative AI tools were used to assist with drafting and language editing. The authors reviewed and verified all content, numerical values, and figures and take full responsibility for the manuscript.

References

- [1] Wu, X., Li, W., Hong, D., Tao, R., Du, Q.: Deep Learning for Unmanned Aerial Vehicle-Based Object Detection and Tracking: A survey. *IEEE Geoscience and Remote Sensing Magazine* **10**(1), 91–124 (2022) <https://doi.org/10.1109/mgrs.2021.3115137>
- [2] Zhu, P., Wen, L., Du, D., Bian, X., Fan, H., Hu, Q., Ling, H.: Detection and Tracking Meet Drones Challenge. *IEEE Transactions on Pattern Analysis and Machine Intelligence*

- 44(11), 7380–7399 (2022) <https://doi.org/10.1109/tpami.2021.3119563>
- [3] Du, D., Qi, Y., Yu, H., Yang, Y., Duan, K., Li, G., Zhang, W., Huang, Q., Tian, Q.: The Unmanned Aerial Vehicle Benchmark: Object Detection and Tracking. In: *Computer Vision – ECCV 2018. Lecture Notes in Computer Science*, pp. 375–391. Springer, Cham (2018). https://doi.org/10.1007/978-3-030-01249-6_23
 - [4] Cheng, G., Yuan, X., Yao, X., Yan, K., Zeng, Q., Xie, X., Han, J.: Towards Large-Scale Small Object Detection: Survey and Benchmarks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 1–20 (2023) <https://doi.org/10.1109/tpami.2023.3290594>
 - [5] Bewley, A., Ge, Z., Ott, L., Ramos, F., Upcroft, B.: Simple online and realtime tracking. In: *2016 IEEE International Conference on Image Processing (ICIP)*, pp. 3464–3468. IEEE, Phoenix, AZ, USA (2016). <https://doi.org/10.1109/icip.2016.7533003>
 - [6] Zhang, Y., Sun, P., Jiang, Y., Yu, D., Weng, F., Yuan, Z., Luo, P., Liu, W., Wang, X.: ByteTrack: Multi-object Tracking by Associating Every Detection Box. In: *Computer Vision – ECCV 2022. Lecture Notes in Computer Science*, pp. 1–21. Springer, Cham (2022). https://doi.org/10.1007/978-3-031-20047-2_1
 - [7] Chin, T.-W., Ding, R., Marculescu, D.: AdaScale: Towards Real-time Video Object Detection Using Adaptive Scaling. *arXiv preprint arXiv:1902.02910* (2019)
 - [8] Xu, R., Zhang, C.-l., Wang, P., Lee, J., Mitra, S., Chatterji, S., Li, Y., Bagchi, S.: ApproxDet: Content and Contention-Aware Approximate Object Detection for Mobiles. *arXiv preprint arXiv:2010.10754* (2020)
 - [9] Ma, L.V., Hussain, M.I., Park, J., Kim, J., Jeon, M.: Adaptive Confidence Threshold for ByteTrack in Multi-Object Tracking. In: *2023 12th International Conference on Control, Automation and Information Sciences (ICCAIS)*, pp. 370–374. IEEE (2023). <https://doi.org/10.1109/iccais59597.2023.10382403>
 - [10] Shim, K., Ko, K., Hwang, J., Kim, C.: AdapTrack: Adaptive Thresholding-Based Matching for Multi-Object Tracking. In: *2024 IEEE International Conference on Image Processing (ICIP)*, pp. 2222–2228. IEEE (2024). <https://doi.org/10.1109/icip51287.2024.10648139>
 - [11] Wojke, N., Bewley, A., Paulus, D.: Simple online and realtime tracking with a deep association metric. In: *2017 IEEE International Conference on Image Processing (ICIP)*, pp. 3645–3649. IEEE, Beijing, China (2017). <https://doi.org/10.1109/icip.2017.8296962>
 - [12] Bergmann, P., Meinhardt, T., Leal-Taixe, L.: Tracking Without Bells and Whistles. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 941–951 (2019)
 - [13] Meinhardt, T., Kirillov, A., Leal-Taixe, L., Feichtenhofer, C.: TrackFormer: Multi-Object Tracking With Transformers. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 8844–8854 (2022)
 - [14] Dendorfer, P., Osep, A., Milan, A., Schindler, K., Cremers, D., Reid, I., Roth, S., Leal-Taixe, L.: MOTChallenge: A Benchmark for Single-Camera Multiple Target Tracking. *arXiv preprint arXiv:2010.07548* (2020) [2010.07548](https://doi.org/10.1109/cvpr52688.2022.00867)
 - [15] Liu, S., Li, X., Lu, H., He, Y.: Multi-Object Tracking Meets Moving UAV. In: *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 8866–8875. IEEE (2022). <https://doi.org/10.1109/cvpr52688.2022.00867>

- [16] Dai, P., Chen, L., Lian, Y., Xie, L.: LSO-YOLO: a lightweight real-time object detection network for UAV small-object detection. *Journal of Real-Time Image Processing* **23**(5), 176 (2026) <https://doi.org/10.1007/s11554-026-01977-y>
- [17] Qi, L., Wang, L., Jiang, S.: OATrack: A Quality-Gated Progressive Association Framework with a YOLO Detection Cache for UAV Small-Object Multi-Object Tracking. *Sensors* **26**(16), 5222 (2026) <https://doi.org/10.3390/s26165222>
- [18] Han, Y., Huang, G., Song, S., Yang, L., Wang, H., Wang, Y.: Dynamic Neural Networks: A Survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence* **44**(11), 7436–7456 (2022) <https://doi.org/10.1109/tpami.2021.3117837>
- [19] Yang, L., Han, Y., Chen, X., Song, S., Dai, J., Huang, G.: Resolution Adaptive Networks for Efficient Inference. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2369–2378 (2020)
- [20] Jiang, J., Ananthanarayanan, G., Bodik, P., Sen, S., Stoica, I.: Chameleon: Scalable Adaptation of Video Analytics. In: *Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication*, pp. 253–266. ACM, New York, NY, USA (2018). <https://doi.org/10.1145/3230543.3230574>
- [21] Akiba, T., Sano, S., Yanase, T., Ohta, T., Koyama, M.: Optuna: A Next-generation Hyperparameter Optimization Framework. In: *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 2623–2631. ACM, Anchorage, AK, USA (2019). <https://doi.org/10.1145/3292500.3330701>
- [22] Deb, K., Pratap, A., Agarwal, S., Meyarivan, T.: A Fast and Elitist Multiobjective Genetic Algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation* **6**(2), 182–197 (2002) <https://doi.org/10.1109/4235.996017>
- [23] Luiten, J., Ošep, A., Dendorfer, P., Torr, P., Geiger, A., Leal-Taixé, L., Leibe, B.: HOTA: A Higher Order Metric for Evaluating Multi-object Tracking. *International Journal of Computer Vision* **129**(2), 548–578 (2021) <https://doi.org/10.1007/s11263-020-01375-2>
- [24] Efron, B., Tibshirani, R.J.: *An Introduction to the Bootstrap*. Springer, New York (1993). <https://doi.org/10.1007/978-1-4899-4541-9>
- [25] Jocher, G., Chaurasia, A., Qiu, J.: Ultralytics YOLOv8. <https://github.com/ultralytics/ultralytics>. Version 8.0.0 (2023)
- [26] Liu, K., Jin, S., Fu, Z., Chen, Z., Jiang, R., Ye, J.: Uncertainty-aware Unsupervised Multi-Object Tracking. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 9996–10005 (2023)
- [27] Ge, Z., Liu, S., Wang, F., Li, Z., Sun, J.: YOLOX: Exceeding YOLO Series in 2021. *arXiv preprint arXiv:2107.08430* (2021) [2107.08430](https://arxiv.org/abs/2107.08430)
- [28] Liu, Z., Wang, X., Wang, C., Liu, W., Bai, X.: SparseTrack: Multi-Object Tracking by Performing Scene Decomposition Based on Pseudo-Depth. *IEEE Transactions on Circuits and Systems for Video Technology* **35**(5), 4870–4882 (2025) <https://doi.org/10.1109/TCSVT.2024.3524670>
- [29] Jocher, G., Qiu, J.: Ultralytics YOLO11. <https://github.com/ultralytics/ultralytics>. Version 11.0.0 (2024)